# 自 然 语 言 处 理

## 第3章 文本向量化

授课教师：李芳芳
联系方式：lifangfang@csu.edu.cn

# 本章教学内容

## 文本向量化

### 本章内容一览表

| 每小节概要 | 每小节知识点 | 重难点 |
| --- | --- | --- |
| 第1小节　前景概要 | 为什么要进行文本向量化? | 否 |
| 第2小节　离散表示：词袋模型 | 四种词袋模型：Onehot、TF、TF-IDF、Ngram | **是** |
| 第3小节　分布式表示：词嵌入 | 为什么要进行词嵌入? | 否 |
| 第4小节　NNLM、Word2Vec | NNLM、Skip-gram、CBOW | **是** |
| 第5小节　ELMO模型 | ELMO进行文本向量化的原理、优缺点 | **是** |
| 第6小节　GPT模型 | GPT进行文本向量化的原理、优缺点 | **是** |
| 第7小节　课堂小结 | 内容小结 | 否 |

# 本章教学内容

## 文本向量化思维导图



# 本章教学内容

## 文本向量化

### 本章内容一览表

| 每小节概要 | 每小节知识点 | 重难点 |
| --- | --- | --- |
| 第1小节 前景概要 | 为什么要进行文本向量化? | 否 |
| 第2小节 离散表示：词袋模型 | 四种词袋模型：Onehot、TF、TF-IDF、Ngram | **是** |
| 第3小节 分布式表示：词嵌入 | 为什么要进行词嵌入? | 否 |
| 第4小节 NNLM、Word2Vec | NNLM、Skip-gram、CBOW | **是** |
| 第5小节 ELMO模型 | ELMO进行文本向量化的原理、优缺点 | **是** |
| 第6小节 GPT模型 | GPT进行文本向量化的原理、优缺点 | **是** |
| 第7小节 课堂小结 | 内容小结 | 否 |

# 1 前景概要

## 为什么要向量化?

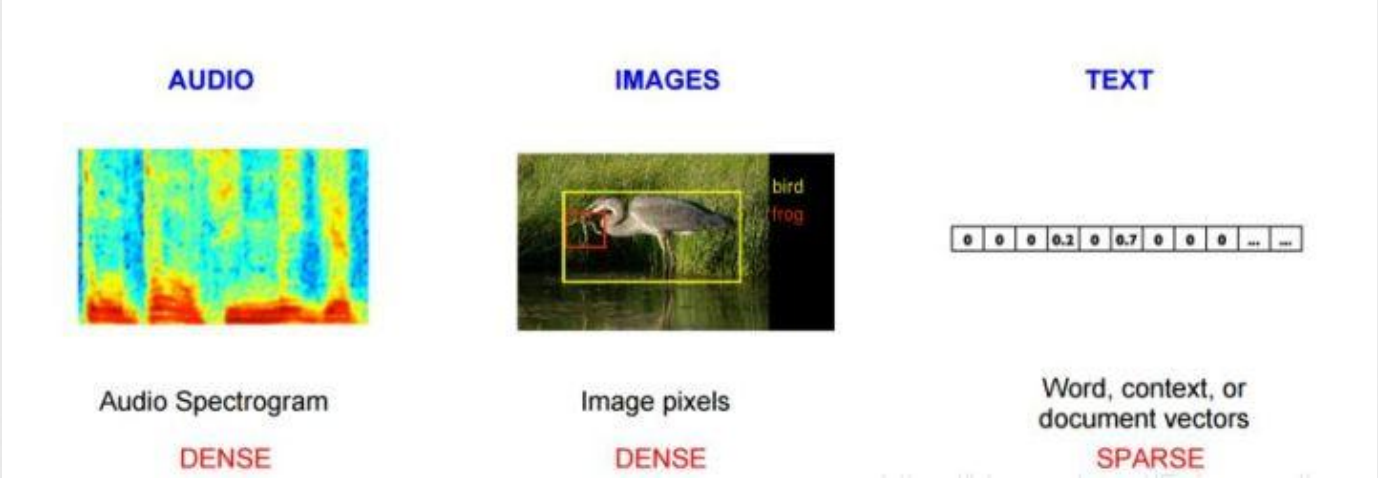
- 使计算机完成情感分析、实体识别、机器翻译、文本生成等等这些任务都是NLP的目标。
- 但完成这些任务的前提：是把语言表示这一关过了。
- 如何让语言表示成为计算机能够处理的数据类型?

**——文本向量化**

# 1 前景概要

## 为什么要向量化?

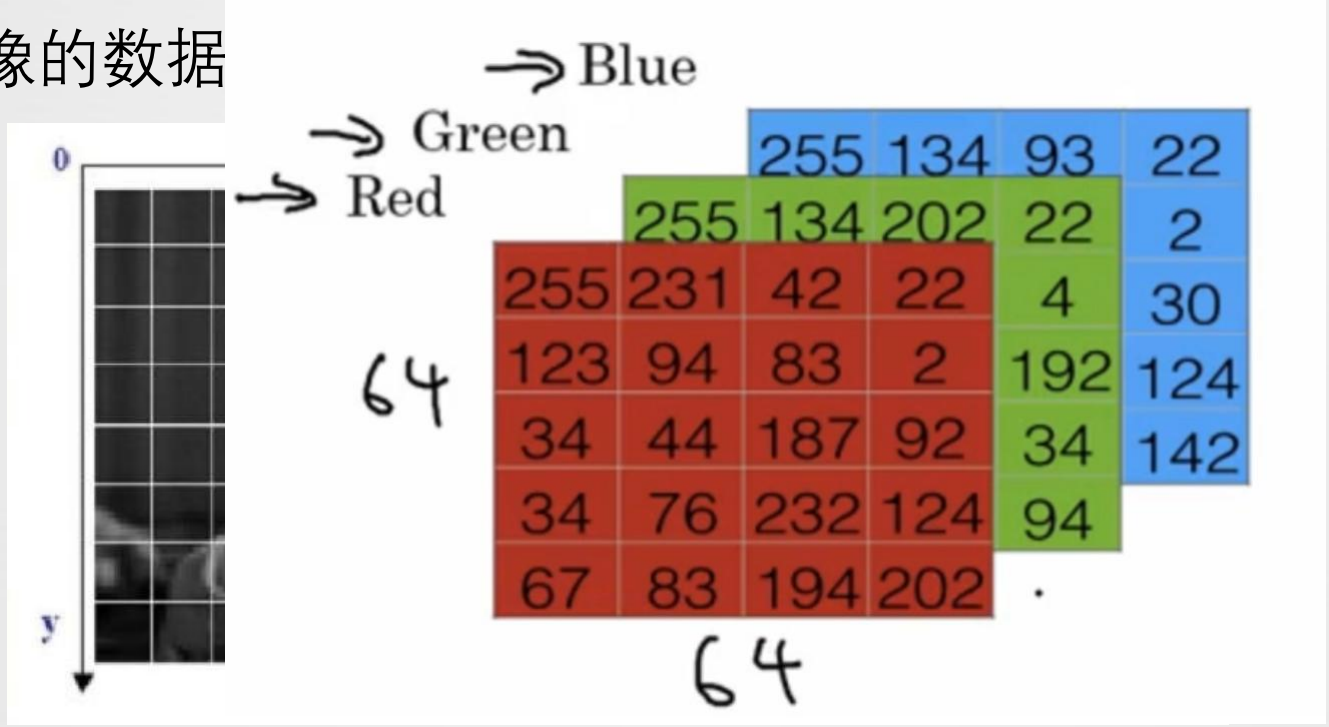
- 我们先来看看图像和语音是怎么表示数据的:



# 1 前景概要

## 为什么要向量化?

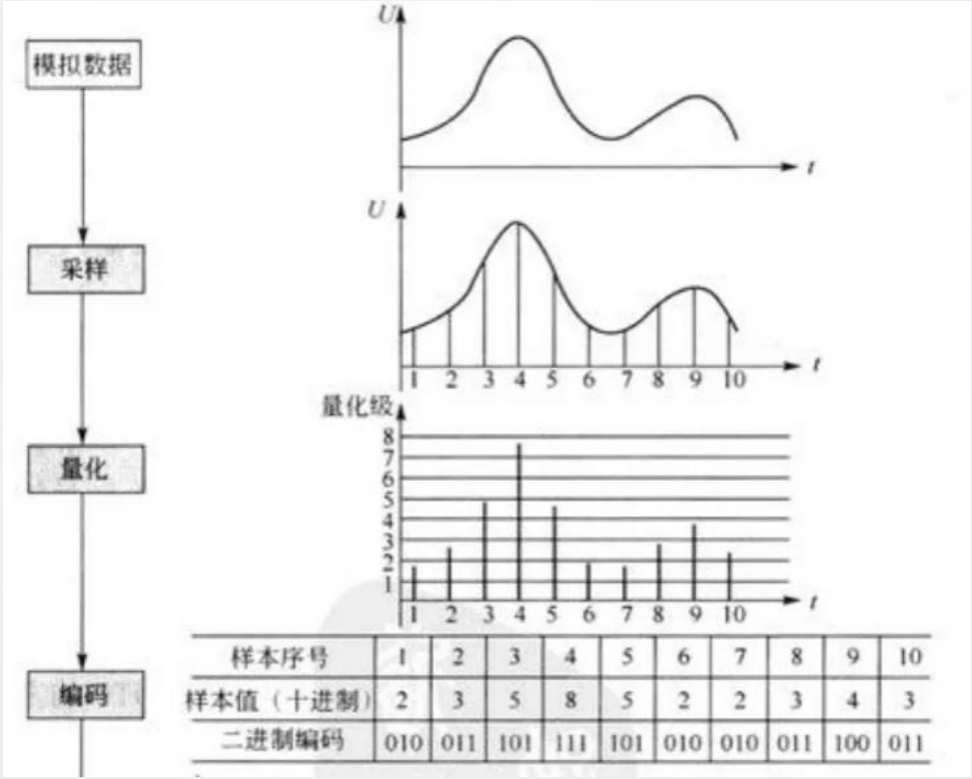
- 图像的数据



# 1 前景概要

## 为什么要向量化?

- 语音的数据表示:

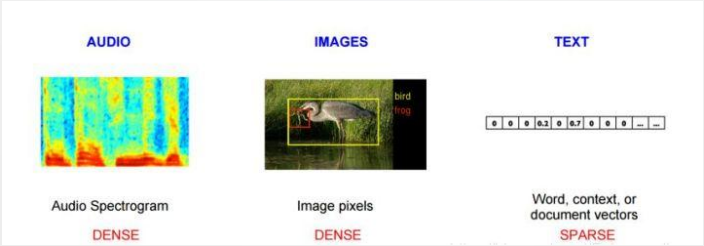



| 样本序号 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| 样本值（十进制） | 2 | 3 | 5 | 8 | 5 | 2 | 2 | 3 | 4 | 3 |
| 二进制编码 | 010 | 011 | 101 | 111 | 101 | 010 | 010 | 011 | 100 | 011 |

# 1 前景概要

## 为什么要向量化?

- 在语音中，数据表示为音频频谱序列向量所构成的matrix;
- 在图像中，数据表示为图片的像素构成的matrix;
- **要将文本数据表示为向量，却不像语音和图像数据可以直接比较自然地转换为低级数据形式。**



# 1 前景概要

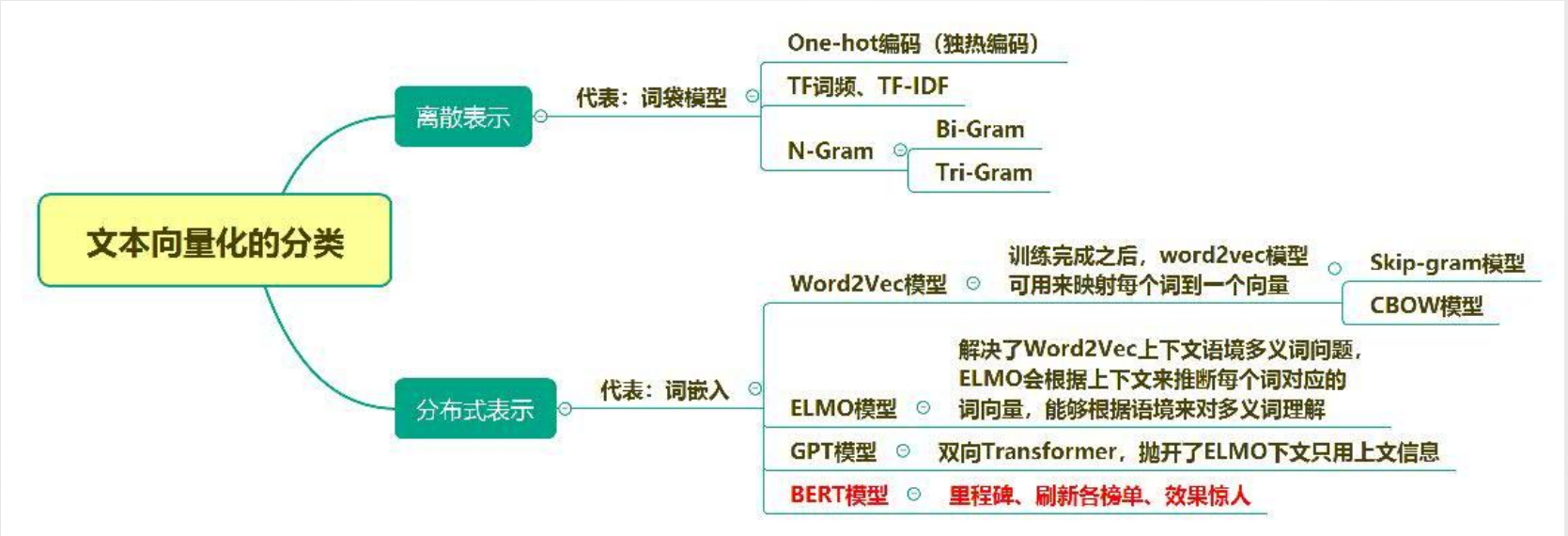
## 为什么要向量化?

- 所以，自然语言（文本）为什么要向量化呢?

文本表示是自然语言处理中的基础工作，文本表示的好坏直接影响到整个自然语言处理系统的性能。**文本向量化**就是将文本表示成一系列能够表达文本语义的向量，是文本表示的一种重要方式。

# 1 前景概要

## 文本的向量化有哪些方法?



# 本章教学内容

## 文本向量化

### 本章内容一览表

| 每小节概要 | 每小节知识点 | 重难点 |
| --- | --- | --- |
| 第1小节　前景概要 | 为什么要进行文本向量化? | 否 |
| 第2小节　离散表示：词袋模型 | 四种词袋模型：Onehot、TF、TF-IDF、Ngram | **是** |
| 第3小节　分布式表示：词嵌入 | 为什么要进行词嵌入? | 否 |
| 第4小节　NNLM、Word2Vec | NNLM、Skip-gram、CBOW | **是** |
| 第5小节　ELMO模型 | ELMO进行文本向量化的原理、优缺点 | **是** |
| 第6小节　GPT模型 | GPT进行文本向量化的原理、优缺点 | **是** |
| 第7小节　课堂小结 | 内容小结 | 否 |

# 2 离散表示：词袋模型

## 词袋模型

- 经过分词处理后的文本以词串的形式存在，但计算机仍无法理解词，只有将词转化为可计算的数之后，计算机才可能对其进行处理从而达到理解的目的。
- 也就是说，**将词转变为可计算的数，是计算的前提。**

# 2 离散表示：词袋模型

## 词袋模型

- 文本表示的好坏直接影响到整个自然语言处理系统的性能。文本向量化就是将文本表示成一系列能够表达文本语义的向量。

自然 语言 处理



[0.2629, 0.3762, 0.7943]

## 2 离散表示：词袋模型

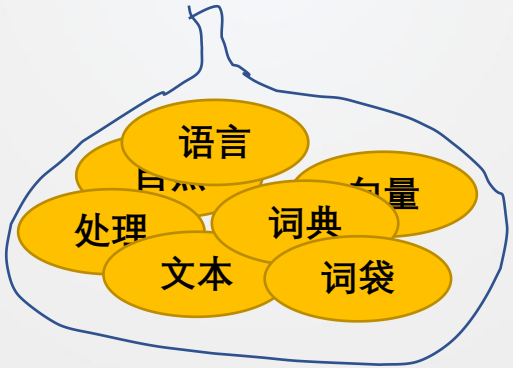
**思考**

# 如何将词串转化为向量？（转换规则）

# 2 离散表示：词袋模型

## 词袋模型（Bag of Words）

- 在进行文本向量化的时候，可以先把所有的词都装入一个袋子（**词典**），之后把每个词都赋予一个**独特的值**，使得词之间能够进行区分，这就是词袋模型的基本思路。



# 2 离散表示：词袋模型

## 词袋模型（Bag of Words）

- 这里有两个关键的地方。第一个重点是**词典**的构建，要从足够充分的训练语料中得到词典。有了足够大、足够完善的词典，后续才能将所有的词都进行向量化表示。
- 第二个是如何**计算每个词的值**，也就是得到每个词的向量表示。

**本节将会讲述三种计算方法：One-Hot法、TF法、TF-IDF法**

# 2 离散表示：词袋模型

## One-Hot编码

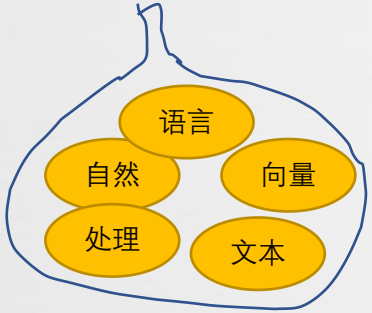
- 一个非常简单的表示法就是，统计各个词在文本中是否出现，出现则表示为1，未出现则表示为0。
- 这就是One-Hot编码，也叫作独热编码，比如词典中一共有N个词，向量的维度就是N维，由于每个词都是独特的，于是，该词只有在它所对应一个维度值为1，其余维都是0。

# 2 离散表示：词袋模型

## One-Hot编码

$$w_j = \begin{cases} 1, & \text{文本含有词项} j \\ 0, & \text{文本不含词项} j \end{cases}$$

词典：




每个词的词向量：

| 词 | 词向量 |
| --- | --- |
| 自然 | [1,0,0,0,0] |
| 语言 | [0,1,0,0,0] |
| 文本 | [0,0,1,0,0] |
| 处理 | [0,0,0,1,0] |
| 向量 | [0,0,0,0,1] |

# 2 离散表示：词袋模型

## One-Hot编码

- 每个词的向量已经知道了，若要表示一个句子，使用的规则是相同的：若该语句中出现某词，则该词对应位置为1，未出现则为0。

# 2 离散表示：词袋模型

## One-Hot编码

每个词的向量：

| 词 | 词向量 |
| --- | --- |
| 自然 | [1,0,0,0,0] |
| 语言 | [0,1,0,0,0] |
| 文本 | [0,0,1,0,0] |
| 处理 | [0,0,0,1,0] |
| 向量 | [0,0,0,0,1] |

待处理语句：

“自然语言处理”



该句的向量表示：

[1,1,0,1,0]

于是可以看出，One-Hot表示法考虑到了有哪些词在文本中出现，用出现的词来表示文本的语义。

# 2 离散表示：词袋模型

## 词频(Term Frequency)

- One-Hot编码只统计了词是否出现，而无法表示词的重要程度。于是很容易想到，根据词频TF来判断词的重要性。即一个词（非停用词）在文本中出现的次数越多，那么该词对于文本来说就越重要。
- TF表示法的规则是，词语序列中出现的词语的数值为该词在文本中的频次，未出现则为0。用数学式表达为：

$$w_j = \begin{cases} count(t_j), \text{文本含有词项} j \\ 0, \text{文本不含词项} j \end{cases}$$

其中，$t_j$表示词语$j$，$count(t_j)$表示词语$j$在文本中出现的次数。

# 2 离散表示：词袋模型

## 词频(Term Frequency)

例如待处理语句：“自然 语言 处理 处理”

词袋：　　[自然，语言，文本，处理，向量]

该句的TF表示：[1,　　1,　　0,　　2,　　0]

可以看出，TF表示法除了考虑到哪些词在文本中出现外，还考虑了词出现的频次，用词语的频次来突出文本主题进而表示文本的语义

## 词频(Term Frequency)

- 虽然TF表示法通过统计词频来表示词的重要性，但有些时候这种表示并不准确。
- 比如，在某一篇文章中发现“中国”，“蜜蜂”，“养殖”这三个词的出现频率名列前茅，但这三个词的重要性显然不一样。因为“中国”是很常见的词，相对而言，“蜜蜂”和养殖不那么常见，所以在这篇文章中，有理由认为“蜜蜂”和“养殖”的重要性程度要大于“中国”。

# 2 离散表示：词袋模型

## 词频(Term Frequency)

- 虽然TF表示法通过统计词频来表示词的重要性，但有些时候这种表示并不准确。
- 比如在某一篇文章中发现**“中国”，“蜜蜂”，“养殖”**这三个词的出现频率名列前茅，但这三个词的重要性显然不一样。因为“中国”是很常见的词，而“蜜蜂”和“养殖”不那么常见，所以在这篇文章中，有理由认为“蜜蜂”和“养殖”的重要程度大于“中国”。

# 2 离散表示：词袋模型

## 词频(Term Frequency)

- 所以需要一个重要性的调整系数，来衡量一个词是不是常见词。
- **如果某个词比较少见，但是在某文本中多次出现，那么它的重要程度则很高。**

## TF-IDF 表示法

- 于是，要在词频的基础上，对每个词分配一个“重要性”的**权重**。
- 最常见的词（比如“的”，“是”）要给予最小的权重，而较为常见的词（比如“中国”）给予较小的权重，而较少出现的词（比如“蜜蜂”）给予较大的权重。
- 这个权重叫做**逆文档频率**（**Inverse Document Frequency，IDF**），它的大小与一个词的常见程度成反比。

# 2 离散表示：词袋模型

## TF-IDF 表示法

- 知道了词频（TF）和逆文档频率（IDF）之后，将这两个值相乘，就得到了一个词的**TF-IDF**值。某个词对于文本的重要性程度越高，它的TF-IDF值就越大。未出现的词的TF-IDF值为0。
- TF-IDF值的计算式为：

# 2 离散表示：词袋模型

## TF-IDF 表示法

- 1、词频TF = 某词在文档中的出现次数
- 2、考虑到文档有长短之分，为了便于不同文档的比较，进行词频标准化。词频TF = 某词在文档中的出现次数 / 文档的总次数

  或 = 某词在文档中的出现次数 / 文档中所有词的出现次数最大值
- 3、计算逆文档频率IDF = log(语料库文档总数 / (包含该词的文档数 + 1))
- 4、计算TF-IDF值 = 词频TF × 逆文档频率IDF

## TF-IDF 表示法

- 例如：一篇文本有1000个词，“中国”，“蜜蜂”，“养殖”各出现20词，于是这三个词的词频（TF）都是0.02。
- 假定语料库中总计有100万个文本，包含“中国”文本有34.7万个，包含“蜜蜂”的文本有0.29万个，包含“养殖”的文本有0.71个。于是这三个词的IDF和TF-IDF值分别为：

# 2 离散表示：词袋模型

## TF-IDF 表示法

| | 包含该词的文本数（万） | IDF | TF-IDF |
|---|---|---|---|
| 中国 | 34.7 | 0.4597 | 0.09194 |
| 蜜蜂 | 0.29 | 2.5376 | 0.50752 |
| 养殖 | 0.71 | 2.1487 | 0.42974 |

## TF-IDF 表示法

- 从上表可见，“蜜蜂”的TF-IDF值最高，“养殖”其次，“中国”最低（如果还计算“的”字的TF-IDF，那将是一个极其接近0的值）。所以，如果只选择一个词，"蜜蜂"就是这篇文章的关键词。

## TF-IDF 表示法

- 可以发现，TF-IDF表示法除了考虑有哪些词在文本中出现外，还考虑了词出现的频次以及词语在所有文本上的频率倒数。
- 用出现词语的频次来突出文本主题，用出现词语的逆文档频率来突出文档的独特性，进而表示文本的语义。

# 2 离散表示：词袋模型

## TF-IDF 表示法

- 优点：
  - ✓ 简单快速且能达到一定的效果
  - ✓ 自身能够有效应对停用词

- 缺点：
  - ✓ **未利用词的顺序信息**
  - ✓ **语义丢失（近义词无法体现、忽略词与词之间的依赖性）**

# 2 离散表示：词袋模型

## N-gram

- **问题：假设我们已知一个长度为m个词的句子中的每一个词的概率，如何表示一整个句子的概率呢?**
- 最直接的计算思路：
- $P(w_1 w_2 \ldots w_m) = P(w_1) * P(w_2|w_1) * P(w_3|w_1, w_2) \ldots P(w_m|w_1, \ldots, w_{m-1})$

⇧ 整个句子的概率

⇧ 第一个词的概率

⇧ 已知第一个词的条件下第二个词的概率

⇧ 已知第一、二个词的条件下第三个词的概率

# 回顾：贝叶斯定义

## 贝叶斯定理

- 这里先解释什么是条件概率、先验概率、后验概率

$P(A|B)$ **在事情B发生的条件下A发生的条件概率，其求解公式为**

$$P(A|B) = \frac{P(AB)}{P(B)}$$

P(A)：表示在没有训练数据前假设A拥有的初始概率，被称为A的先验概率.
机器学习中我们关心的是P(B|A)，即给定A时B的成立的概率，称为B的后验概率。

# 回顾：贝叶斯定义

## 贝叶斯定理

- 下面不加证明给出贝叶斯定理公式

$$P(B|A)=\frac{P(A|B)P(B)}{P(A)}$$

- 贝叶斯定理的意义在于，我们在生活中经常遇到这种情况：
- 我们可以很容易直接得出P(A|B)，P(B|A)则很难直接得出，
- 但我们更关心P(B|A)，贝叶斯定理为我们打通从P(A|B)获得P(B|A)的道路。

# 回顾：贝叶斯定义

## 贝叶斯定理

- 贝叶斯定理的解释

$$P(B|A)=\frac{P(A|B)P(B)}{P(A)}$$

- P(B|A)随着P(B)和P(A|B)的增长而增长，随着P(A)的增长而减少，即如果A独立于B时被观察到的可能性越大，那么B对A的支持度越小。

# 2 离散表示：词袋模型

## N-gram

- **问题：假设我们已知一个长度为m个词的句子中的每一个词的概率，如何表示一整个句子的概率呢?**
- $P(w_1 w_2 \ldots w_m) = P(w_1) * P(w_2|w_1) * P(w_3|w_1, w_2) \ldots P(w_m|w_1, \ldots, w_{m-1})$
- 例子：计算I love natural language processing中$P(w_2|w_1)$的概率

$$P(w_2|w_1) = P(\text{love}|I) = \frac{P(I, love)}{P(I)} \approx \frac{count(I, love)}{count(I)}$$

$\longrightarrow$ {I love}共同出现的次数

$\longrightarrow$ {I} 出现的次数

**思考：从这个例子中可以看出这个方法存在什么问题?**

# 2 离散表示：词袋模型

## N-gram

- 参数空间过大：在最后一步计算$P(w_m|w_1,\ldots,w_{m-1})$的参数量就达到了$O(n)$

- 数据稀疏严重：越到后面的步骤出现的词组合就越趋近于不存在。

**改进方式：N-gram!**

# 2 离散表示：词袋模型

## N-gram

- 马尔科夫假设（Markov Assumption）：一个词的出现仅与它之前的若干个词有关。
- 在自然语言处理中，如果假定一个词的出现仅依赖于它前面出现的n个词，那么我们则称之为 N-gram。N可以取很高，但是没有必要，考虑极限情况时N-gram就退化成了刚才那种形式，一般取二元(Bi-gram)或者三元(Tri-gram)。
- N-gram中的N也可以理解为一个长度为N的滑动窗口。

# 2 离散表示：词袋模型

## N-gram

- **马尔科夫假设（Markov Assumption）：一个词的出现仅与它之前的若干个词有关。**

- **bigram:**假设下一个词的出现依赖于它前面的一个词

$$P(w_1 w_2 ... w_m) = P(w_1) * P(w_2|w_1) * P(w_3|w_2) ... P(w_m|w_{m-1})$$

- **trigram:**假设下一个词的出现依赖于它前面的两个词。

$$P(w_1 w_2 ... w_m) = P(w_1) * P(w_2|w_1) * P(w_3|w_1, w_2) ... P(w_m|w_{m-2}, w_{m-1})$$

# 2 离散表示：词袋模型

## N-gram

- 回到最开始的问题：假设我们已知一个长度为m个词的句子中的每一个词的概率，如何表示一整个句子的概率呢?

I love natural language processing

⇩ Bi-gram

{I love} 、{love natural}、 {natural language}、 {language processing}

# 2 离散表示：词袋模型

## N-gram

- **回到最开始的问题：假设我们已知一个长度为m个词的句子中的每一个词的概率，如何表示一整个句子的概率呢?**

$$P(w_1 w_2 \ldots w_m) = P(w_1) * P(w_2|w_1) * P(w_3|w_1, w_2) \ldots P(w_m|w_1, \ldots w_{m-1})$$

⇩ **Bi-gram**

$$P(w_1 w_2 \ldots w_m) = P(w_1) * P(w_2|w_1) * P(w_3|w_2) \ldots P(w_m|w_{m-1})$$

# 2 离散表示：词袋模型

## N-gram举例

- 假设语料库总词数为13,748词

| I | 3437 |
|---|---|
| want | 1215 |
| to | 3256 |
| eat | 938 |
| Chinese | 213 |
| food | 1506 |
| lunch | 459 |

# 2 离散表示：词袋模型

## N-gram举例

- Bi-gram词汇共现次数如下：

| | I | want | to | eat | Chinese | food | lunch |
|---|---|---|---|---|---|---|---|
| I | 8 | 1087 | 0 | 13 | 0 | 0 | 0 |
| want | 3 | 0 | 786 | 0 | 6 | 8 | 6 |
| to | 3 | 0 | 10 | 860 | 3 | 0 | 12 |
| eat | 0 | 0 | 2 | 0 | 19 | 2 | 52 |
| Chinese | 2 | 0 | 0 | 0 | 0 | 120 | 1 |
| food | 19 | 0 | 17 | 0 | 0 | 0 | 0 |
| lunch | 4 | 0 | 0 | 0 | 0 | 1 | 0 |

# 2 离散表示：词袋模型

## N-gram举例

- P(I want to eat Chinese food)

=P(I)*P(want|I)*P(to|want)*P(eat|to)*P(Chinese|eat)*P(food|Chinese)

=0.25*1087/3437*786/1215*860/3256*19/938*120/213

= 0.000154171

# 2 离散表示：词袋模型

## N-gram

- N越大特征向量（每个词的词向量）的维度就越庞大。

当前词表中词的个数|V|=20,000

| N | 所有可能的n-gram数 |
|---|---|
| 2 | $4 \times 10^8$ |
| 3 | $8 \times 10^{12}$ |
| 4 | $1.6 \times 10^{17}$ |

# 2 离散表示：词袋模型

## N-gram

- 对比：如何用N-gram表示一个句子的特征？词袋是：{自然 语言 文本 处理 向量}

One-Hot

| 词 | 词向量 |
| --- | --- |
| 自然 | [1,0,0,0,0] |
| 语言 | [0,1,0,0,0] |
| 文本 | [0,0,1,0,0] |
| 处理 | [0,0,0,1,0] |
| 向量 | [0,0,0,0,1] |

Bi-gram

| gram | 词向量 |
| --- | --- |
| {自然,自然} | [1,0,0,0,0…] |
| {自然,语言} | [0,1,0,0,0…] |
| {自然,文本} | [0,0,1,0,0…] |
| {自然,处理} | [0,0,0,1,0…] |
| {自然,向量} | [0,0,0,0,1…] |
| …… | 几维? |

# 2 离散表示：词袋模型

## N-gram

- 优点：
  - ✓ 考虑了词之间一定的顺序关系
  - ✓ 通过马尔科夫假设简化了计算
  - ✓ 简单直观，可解释性强
- 缺点：
  - ✓ **缺乏长期依赖，只能表示当前词与前n-1个词的关系**
  - ✓ **随着n的增大，参数空间呈指数增长**
  - ✓ **近义词无法体现**

# 2 离散表示：词袋模型

## 问题?

- 词袋模型这样的离散化最大的问题就是每个词彼此无关。
- 而实际中，词与词之间是有关系的，比如一些近义词“类似”和“相近”。同样地，一句话中的词也是有一种语序关系的。而词袋模型把词当成独立的个体，不能表示词与词之间的相似程度（语义丢失），也不能反映词语之间的语序关系。另外，这样训练的话，如果词典非常大，那么向量的维度也会是一个庞大的数，这会造成维度灾难。

# 2 离散表示：词袋模型

## 问题?

- 离散化表示方法是最早的以词语为基础处理单元的文本向量化方法。这类方法虽然简单易行，但是普遍存在如下三个方面的问题：**维度灾难，无法保留词序信息，存在语义鸿沟。**

- 由于离散化表示方法存在以上不足以及神经网络的再次流行，故近年来的研究重心转移到**利用无标注数据挖掘有价值的信息上来。**

# 2 离散表示：词袋模型

**问题?**

所以需要一种新的表示方法来表示一个词，每个维度不再是0或1，而是一些连续的实数，以表示不同的程度，越接近的数的词义也是越相似的

# 2 离散表示：词袋模型

**解决方法：连续的实数**

| 词 | 索引 | i=1 | i=2 | … | …2 | i=299 | i=300 |
|---|---|---|---|---|---|---|---|
| Glenn | 1 | 0.5445 | 0.7343 | … | … | 0.4324 | 0.32242 |
| 1Q84 | 2 | 0.3232 | 0.3422 | … | … | 0.7674 | 0.1434 |
| Google | 3 | 0.5232 | 0.6774 | … | … | 0.6543 | 0.7674 |
| Amazon | 4 | 0.3232 | 0.3422 | … | … | 0.7674 | 0.1434 |
| … | … | … | … | … | … | … | … |
| … | … | … | … | … | … | … | … |
| … | … | … | … | … | … | … | … |
| … | … | … | … | … | … | … | … |
| … | … | … | … | … | … | … | … |
| people | 10000 | 0.3232 | 0.3422 | … | … | 0.7674 | 0.1434 |
| person | 10001 | 0.362 | 0.3522 | … | … | 0.7545 | 0.1355 |
| … | … | … | … | … | … | … | … |
| … | … | … | … | … | … | … | … |
| … | … | … | … | … | … | … | … |
| … | … | … | … | … | … | … | … |
| … | … | … | … | … | … | … | … |
| zoo | 999,999 | 0.31246 | 0.3422 | … | … | 0.7674 | 0.1434 |
| zoom | 1000000 | 0.45443 | 0.3776 | … | … | 0.23223 | 0.2334 |

词典维度： V×N=1000000×300；每一行代表一个词的词向量

# 本章教学内容

## 文本向量化

### 本章内容一览表

| 每小节概要 | 每小节知识点 | 重难点 |
| --- | --- | --- |
| 第1小节 前景概要 | 为什么要进行文本向量化? | 否 |
| 第2小节 离散表示：词袋模型 | 四种词袋模型：Onehot、TF、TF-IDF、Ngram | **是** |
| 第3小节 分布式表示：词嵌入 | 为什么要进行词嵌入? | 否 |
| 第4小节 NNLM、Word2Vec | NNLM、Skip-gram、CBOW | **是** |
| 第5小节 ELMO模型 | ELMO进行文本向量化的原理、优缺点 | **是** |
| 第6小节 GPT模型 | GPT进行文本向量化的原理、优缺点 | **是** |
| 第7小节 课堂小结 | 内容小结 | 否 |

# 3 分布式表示：词嵌入

## 什么是分布式表示?

- 传统的one-hot 表示仅仅将词符号化，不包含任何语义信息。
- 如何将语义融入到词表示中?
- Harris 在 1954 年提出的分布假说为这一设想提供了理论基础：上下文相似的词，其语义也相似。
- Firth 在 1957 年对分布假说进行了进一步阐述和明确：词的语义由其上下文决定。

# 3 分布式表示：词嵌入

## 什么是分布式表示?

- 到目前为止，基于分布假说的词表示方法，根据建模的不同，主要分为三类：
  - 基于矩阵的分布表示
  - 基于聚类的分布表示
  - 基于神经网络的分布表示
- 尽管这些不同的分布表示方法使用了不同的技术手段，但它们的核心思想都由两部分组成：
  - 1、选择一种方式描述上下文；
  - 2、选择一种模型刻画某个词（目标词）与其上下文之间的关系。

# 3 分布式表示：词嵌入

## 词嵌入

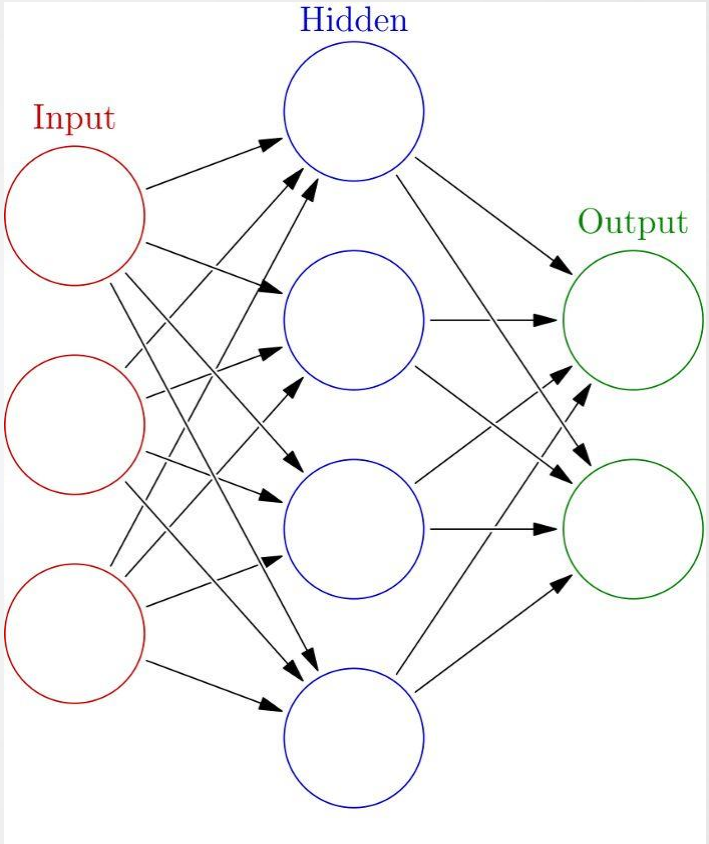
- 基于神经网络的分布式表示一般称为**词向量**或**词嵌入word embedding**
- 神经网络词向量表示技术通过**神经网络技术**对上下文，以及上下文与目标词之间的关系进行建模。由于神经网络较为灵活，这类方法的最大优势在于可以表示复杂的上下文。

**简单复习一下神经网络**

# 神经网络复习

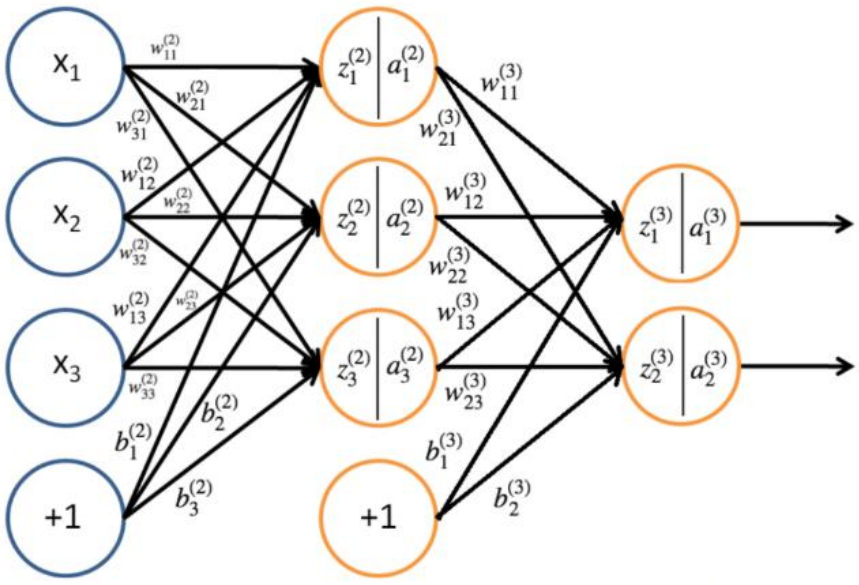
## 前馈神经网络

- 如图所示，神经网络有3层：输入层、隐藏层、输出层，共3、4、2个神经元。
- 输入层：节点数量与数据集的特性数量相同。
- 隐藏层：可以自由选择需要多少节点，并且可以使用多个隐藏层。



# 神经网络复习

## 前向传播例子




隐藏层计算:

$$
\begin{aligned}
z_1^{(2)} &= w_{11}^{(2)}x_1 + w_{12}^{(2)}x_2 + w_{13}^{(2)}x_3 + b_1^{(2)} \\
z_2^{(2)} &= w_{21}^{(2)}x_1 + w_{22}^{(2)}x_2 + w_{23}^{(2)}x_3 + b_2^{(2)} \\
z_3^{(2)} &= w_{31}^{(2)}x_1 + w_{32}^{(2)}x_2 + w_{33}^{(2)}x_3 + b_3^{(2)} \\
a_1^{(2)} &= f(z_1^{(2)}) \\
a_2^{(2)} &= f(z_2^{(2)}) \\
a_3^{(2)} &= f(z_3^{(2)})
\end{aligned}
$$

输出层计算:

$$
\begin{aligned}
z_1^{(3)} &= w_{11}^{(3)}a_1^{(2)} + w_{12}^{(3)}a_2^{(2)} + w_{13}^{(3)}a_3^{(2)} + b_1^{(3)} \\
z_2^{(3)} &= w_{21}^{(3)}a_1^{(2)} + w_{22}^{(3)}a_2^{(2)} + w_{23}^{(3)}a_3^{(2)} + b_2^{(3)} \\
a_1^{(3)} &= f(z_1^{(3)}) \\
a_2^{(3)} &= f(z_2^{(3)})
\end{aligned}
$$

前向传播过程: $\boldsymbol{x} = \boldsymbol{a}^{(1)} \rightarrow \boldsymbol{z}^{(2)} \rightarrow \cdots \rightarrow \boldsymbol{a}^{(L-1)} \rightarrow \boldsymbol{z}^{(L)} \rightarrow \boldsymbol{a}^{(L)} = \boldsymbol{y}$

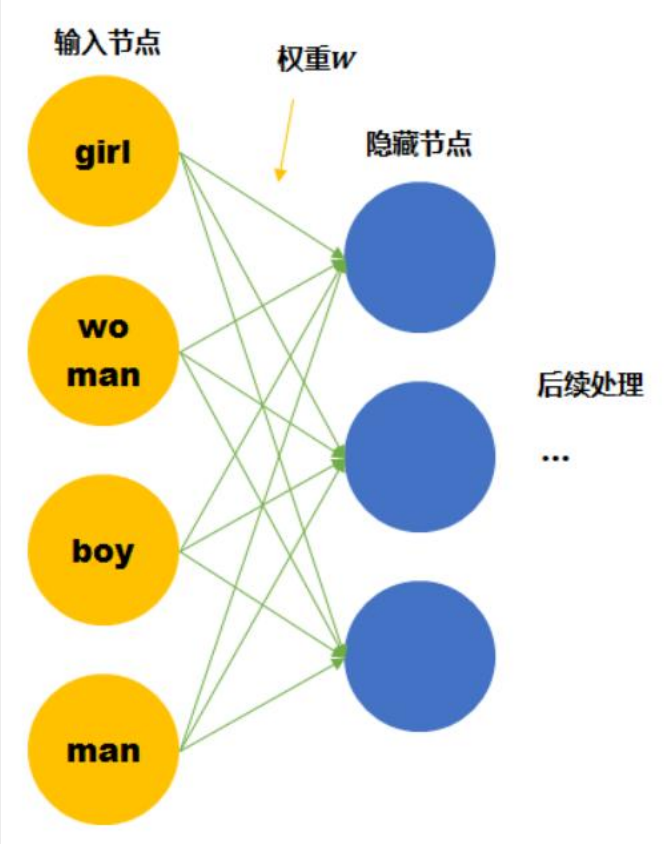
# 3 分布式表示：词嵌入

## 何为词嵌入

- 假设目前有4个词girl，woman，boy，man需要进行文本表示。我们是知道这四个词之间的关系的，但计算机不知道。
- 假设使用One-Hot编码，用神经网络来进行解释的话，在神经网络的输入层中，每个单词被看作一个节点。训练就是要学习连接的权重，如果只看第一层的权重，需要确定4*3个连接的权重，因为每个维度彼此独立，girl的数据不会对其他单词的训练产生任何帮助，所以训练之后单词之间是没有联系的。

# 3 分布式表示：词嵌入

## 何为词嵌入



# 3 分布式表示：词嵌入

## 何为词嵌入

- 我们是可以找出girl，woman，boy，man这四个词之间的联系的，它们之间的区别就在于性别gender和年龄age。假设性别和年龄按照下表进行表示：

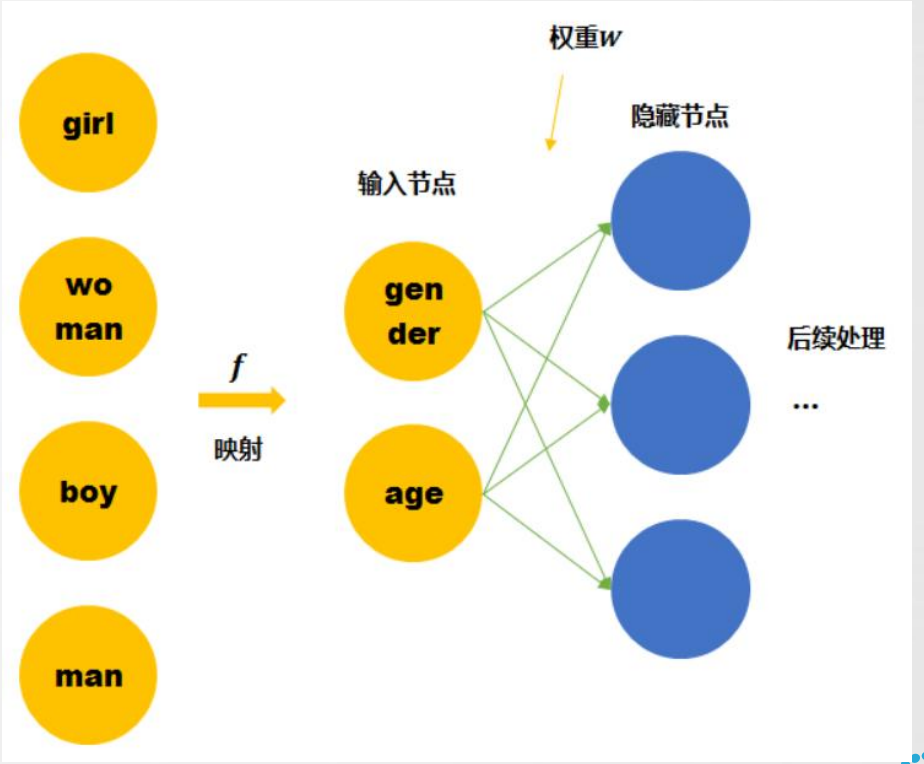
| | 0 | 1 |
|---|---|---|
| gender | female | male |
| age | child | adult |

- 那么girl可以表示为[0,0]，boy可以表示为[1,0]，其他两个也类似。

# 3 分布式表示：词嵌入

## 何为词嵌入

- 这时候再看神经网络需要学习的权重就缩小到了2*3。
- 同时，当输入girl这个词的时候，它是由两个节点编码的，那么与girl共享相同连接线的其他输入也可以被训练到（比如与girl同为female的wowan），这样，词与词之间就有了联系。



# 3 分布式表示：词嵌入

## 何为词嵌入

- Word Embdding就是要将词映射到一个维度较小的向量空间内，
- 进而可以利用普通的统计学方法来研究词与词之间的关系。
- 相对于词袋模型，Word Embedding降低了词向量的维度，减少训练所需数据量。

# 3 分布式表示：词嵌入

## 词嵌入分为哪几种?



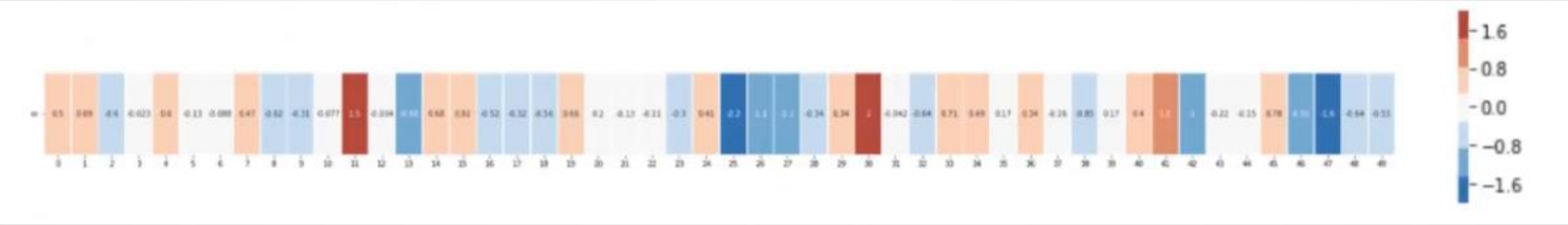
# 3 分布式表示：词嵌入

## 词向量长什么样?

- 假设一个词向量是50维。

```
[ 0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 , -0.61798 , -0.31012 ,
-0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961
, -0.13495 , -0.11476 , -0.30344 , 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 ,
-0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 ,
-1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 , -0.64426 , -0.51042 ]
```

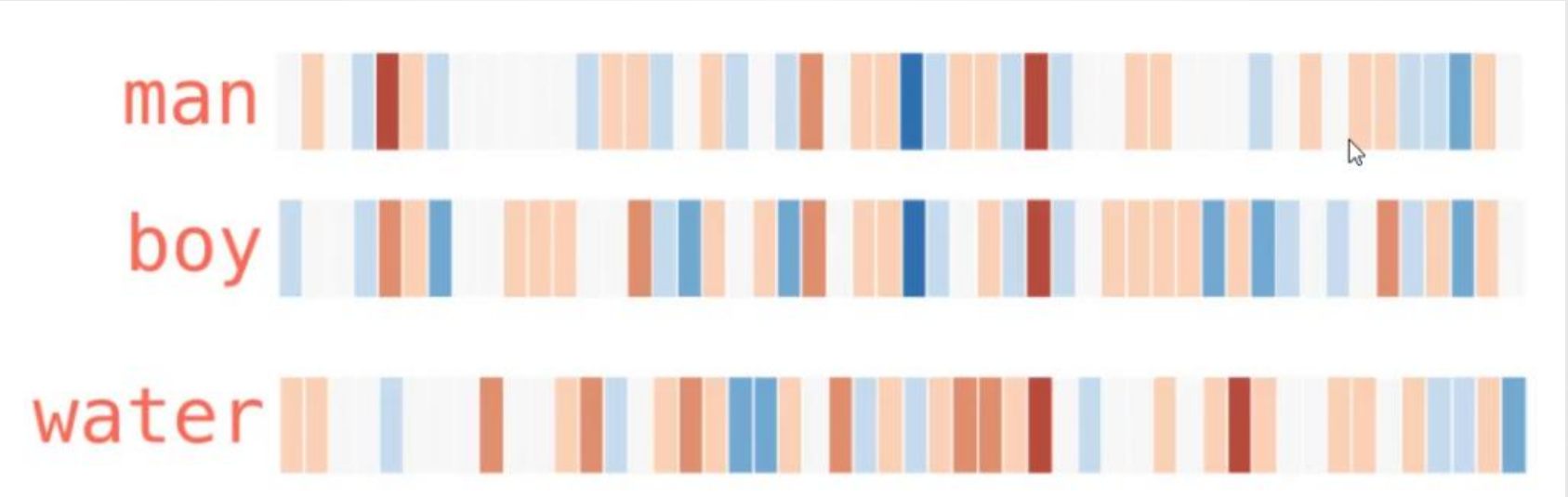
- 我们用热力图中不同的颜色来表示



# 3 分布式表示：词嵌入

## 词向量长什么样?

- 在结果中可以发现，相似的词在词向量表达中比较相似，也就是说明词的特征是有实际意义的。



# 本章教学内容

## 文本向量化

### 本章内容一览表

| 每小节概要 | 每小节知识点 | 重难点 |
| --- | --- | --- |
| 第1小节　前景概要 | 为什么要进行文本向量化? | 否 |
| 第2小节　离散表示：词袋模型 | 四种词袋模型：Onehot、TF、TF-IDF、Ngram | **是** |
| 第3小节　分布式表示：词嵌入 | 为什么要进行词嵌入? | 否 |
| 第4小节　NNLM、Word2Vec | NNLM、Skip-gram、CBOW | **是** |
| 第5小节　ELMO模型 | ELMO进行文本向量化的原理、优缺点 | **是** |
| 第6小节　GPT模型 | GPT进行文本向量化的原理、优缺点 | **是** |
| 第7小节　课堂小结 | 内容小结 | 否 |

# 4 词嵌入之NNLM

## 神经网络语言模型NNLM

- 语言模型（Language Model，LM）是用来计算一个句子的概率的模型，通俗地讲就是用来判断一句话是人话的概率。每个词都要根据前面的所有词计算自己出现的概率，最后把所有词的概率乘起来就是整个句子的概率：

$$P(S) = P(w_1, w_2, \ldots, w_n) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2)\cdots P(w_n|w_1, w_2, \cdots, w_{n-1})$$

- 不过，一般计算一个词出现的概率时，只会考虑该词之前的数个词（马尔克夫假设）（n-gram模型）。

# 4 词嵌入之NNLM

## 神经网络语言模型NNLM

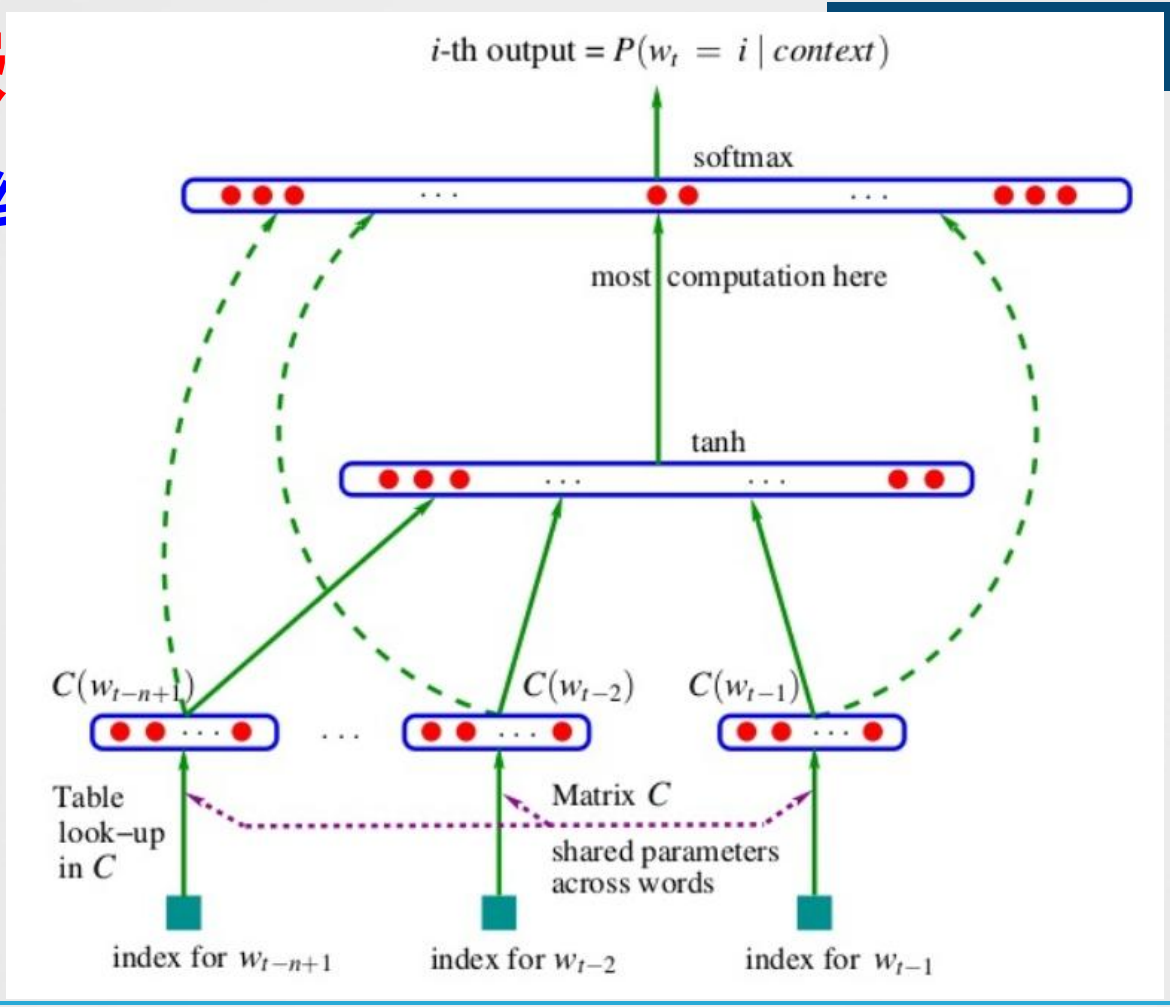
- **神经网络语言模型**（Neural Network Language Model，NNLM）由Yoshua Bengio等人在2003年发表的《A Neural Probabilistic Language Model》中提出。
- 它的任务就是输入句中某个单词$W_t = "Bert"$前面t-1个单词，要求网络正确预测出单词Bert，即最大化：

$$P(W_t = "Bert" | W_1, W_2, ..., W_{t-1}; \theta)$$

**将上述目标表示成一个神经网络优化任务，间接获取每个词的词向量表示**

# 4 词嵌

## 神经网络

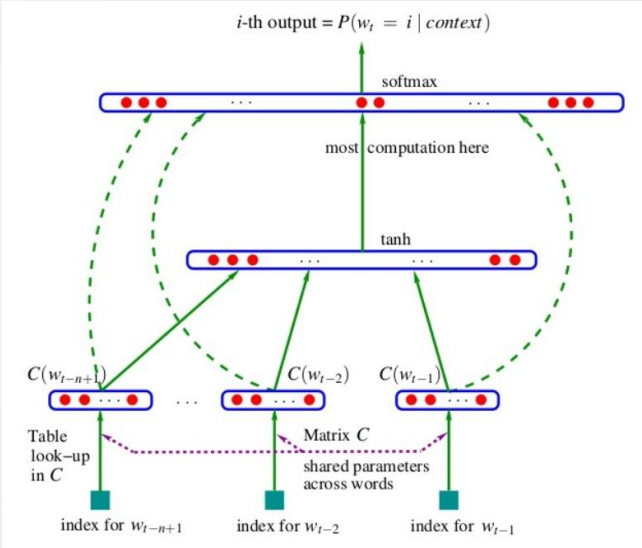

i-th output = P(w_t = i | context)
softmax
most computation here
tanh
C(w_{t-n+1})
C(w_{t-2})
C(w_{t-1})
Table look-up in C
Matrix C
shared parameters across words
index for w_{t-n+1}
index for w_{t-2}
index for w_{t-1}

# 4 词嵌入之NNLM

## 神经网络语言模型NNLM

- 对于一个词序列 $w_1, \ldots, w_T$ ，其中 $w_t \in V$ ， $V$ 是一个很大但是有限的单词集合。我们的目标是训练一个模型，来预测给定词序列的条件下，下一个词是 $w_t$ 的概率：

$$f(w_t, \ldots, w_{t-n+1}) = \hat{P}(w_t | w_1^{t-1})$$



# 4 词嵌入之NNLM

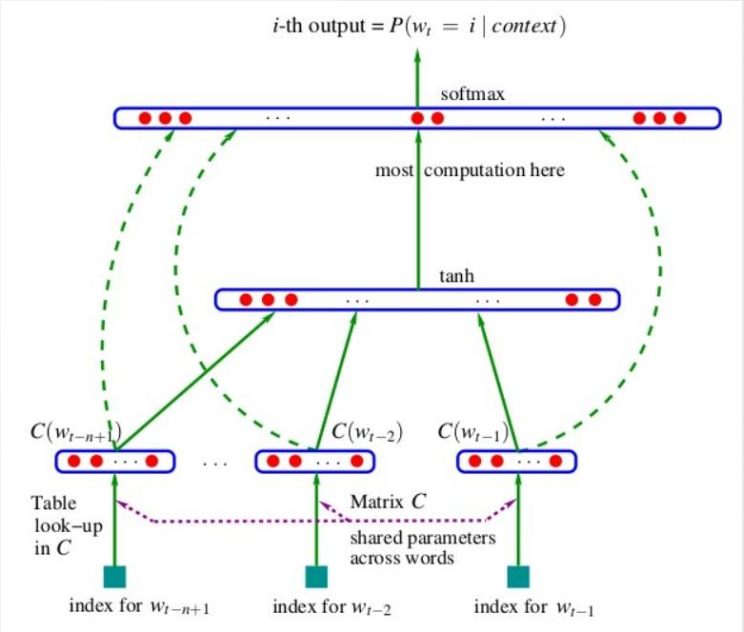
## NNLM模型分析

简单来说，NNML有三个模块：

**1. Embedding Layer：** 模型的这一层将索引向量与embedding矩阵相乘，作为原始单词输入；

**2. Intermediate Layer(s)：** 一层或多层，生成输入层的中间表示，比如，用一个全连接层对前n个词语的词向量的拼接组合进行非线性变换；

**3. Softmax Layer：** 模型的最后一层，生成每个词语的概率分布。



## NNLM模型分析

- **第一个模块**是输入层：前面的单词$W_i$用One-Hot编码（比如：0001000）作为原始单词输入，
- 之后乘以矩阵C后获得每个词的向量$C(i)$（m维）。这里，矩阵C是一个$|V| \times m$的矩阵，$|V|$是词表的大小，m是每个词的维度。
- 之后，把n个词的$C(i)$组合起来（这里的n是参数），得到矩阵$x = (C(1), C(2), ..., C(n-1))$，于是矩阵$x$是$(n-1) \times m$维的矩阵。

# 4 词嵌入之NNLM

## NNLM模型分析

- **第二个模块**是隐藏层，一层或多层，生成输入层的中间表示，
- 在隐藏层中进行如下的计算：

$$o \leftarrow d + Hx; \quad o\text{是长度为}d\text{的向量}$$
$$a \leftarrow tanh(o); \quad a\text{是长度为}h\text{的向量}$$

- 用一个全连接层对前n个词语的词向量进行拼接组合（神经网络的加权）
- 再进行tanh（或其它函数）非线性变换；

# 4 词嵌入之NNLM

## NNLM模型分析

- **第三个模块是输出层，**生成每个词语的概率分布。
- 在输出层中，综合第一层得到的$x$和隐藏层得到的$a$的影响，并使用$softmax$来调整表示概率（为了使得概率和为1，需要应用softmax函数到输出层）。

$$y = b + Wx + Ua$$

$$p(w_t|w_{t-1}, \ldots, w_{t-n+1}) = \frac{e^{y_{w_t}}}{\sum_i e^{y_i}}$$

- 最后得到的$p$是**输出**，**即词$w_t$接在前面n个词之后的概率。**

# 4 词嵌入之NNLM

## NNLM模型分析

- 整个过程中，NNLM不仅根据上下文预测了之后应该跟着的单词是什么，还得到了一个**“副产物”**：

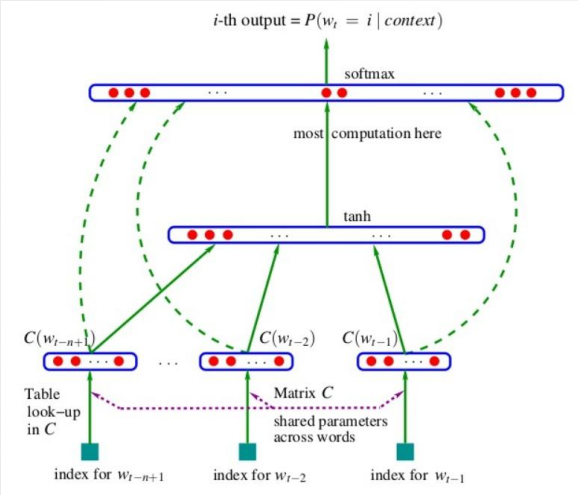
$$\text{矩阵}C = |V| \times m$$

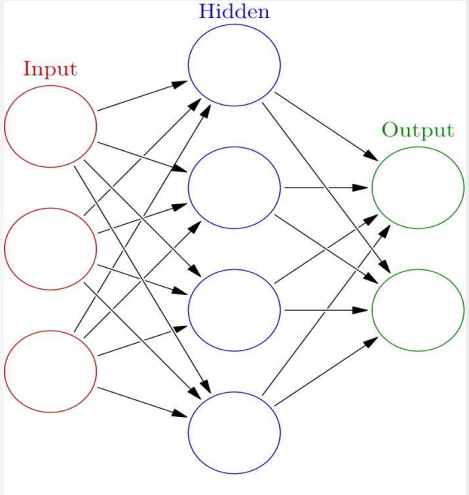
- 这个矩阵中的每一行就是对应单词的Word Embedding值，非常地方便。

NNLM使用更为低维的词向量对上下文进行表示，解决了词袋模型带来的维度爆炸、语义丢失等问题。同时，在相似的上下文语境中，NNLM可以预测出相似的目标词，而传统的模型无法做到这一点。

# 4 词嵌入之NNLM

## NNLM模型VS神经网络模型




**实验结果没有表明直连边是否有效，不过在小样本实验中发现，有直连边可以减少迭代次数，没有直连边的模型泛化能力更好一些。**